TECHNISCHE UNIVERSITÄT WIEN

C Pitfalls

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch, Peter Puschner

2026-10-27

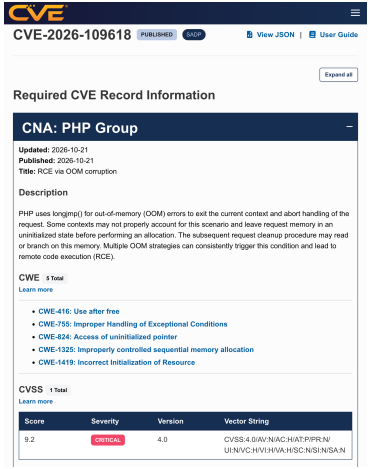
About C...

Andrew Koenig - AT&T Bell Labs (biased?)

> *The C language is like a carving knife: simple, sharp, and extremely useful in skilled hands. Like any sharp tool, C can injure people who don't know how to handle it.*

PHP2Shell (CVE-2026-109618)

- Bad handling of cleanup after OOM allows for remote code execution
- Discovered Aug 2026, disclosed 6 days ago
- Affects about 70% of all websites
- Who knows "how to handle it"?



CVE

CVE-2026-109618 PUBLISHED SADP View JSON | User Guide

Expand all

Required CVE Record Information

CNA: PHP Group

Updated: 2026-10-21
Published: 2026-10-21
Title: RCE via OOM corruption

Description

PHP uses long(jmp() for out-of-memory (OOM) errors to exit the current context and abort handling of the request. Some contexts may not properly account for this scenario and leave request memory in an uninitialized state before performing an allocation. The subsequent request cleanup procedure may read or branch on this memory. Multiple OOM strategies can consistently trigger this condition and lead to remote code execution (RCE).

CWE 5 Total
Learn more

- CWE-416: Use after free
- CWE-755: Improper Handling of Exceptional Conditions
- CWE-824: Access of uninitialized pointer
- CWE-1325: Improperly controlled sequential memory allocation
- CWE-1419: Incorrect Initialization of Resource

CVSS 1 Total
Learn more

Score	Severity	Version	Vector String
9.2	CRITICAL	4.0	CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

Part I

Syntactic Pitfalls

Declarations


```
int x, ((y));          x and y are int
int *i, j;             i is pointer to int, j is int

int f(void);           f is function that returns int
int *g(void);          g is function that returns pointer to int
int (*h)(void);        h is pointer to function that returns int
```

Useful concept: "declaration follows use"

(The declaration tells you how to get a value of a certain type). Examples:

```
int x;          x is an integer
int *i;         *i is an integer
int f(void);    f() (i.e., value obtained by calling f) is an integer
```

Bitwise vs Logical Operators

- && and || are logical operations
- & and | are bitwise operations

```
1  char a = 2, b = 4;
2
3  if (a && b) { foo(); }
4
5  if (a & b) { foo(); }
```

How many times does this piece of code call `foo()`? 1

a:	0	0	0	0	0	0	1	0
b:	0	0	0	0	0	1	0	0
a & b:	0	0	0	0	0	0	0	0

Characters

- On some platforms, char = signed char, on others unsigned!
  - Check `CHAR_MIN` and `CHAR_MAX` in `<limits.h>`
  - All other integer types are signed by default
  - Explicitly declared chars as (un)signed unless you only use them for text
- `''` for character literals
  - Type `int`; just another way to write an integer
  - Value depends on encoding (usually ASCII)
  - Instead, `""` for string literals (type `const char*`)
- Escape sequences with `\` for special characters (e.g., newline `\n`)

```
1  char a1 = 'a';          /* 97 in ASCII */
2  char newline1 = '\n';   /* 32 in ASCII */
3  int test = (a1 == 97) && (newline1 == 32); /* true (if compiler uses ASCII) */
```

Semicolons and if/while/for


```
1  if (x[i] > big);
2    big = x[i];
```

- `big` is not the body of the `if`
- `big` will always be assigned a new value!
- `if`/`while`/`for`... expect a statement/block
- Semicolon following the condition is not a syntax error:
  it is an empty statement ("if (*condition*) do nothing")

Dangling else

- `else` always refers to the nearest `if`
- Don't let indentation fool you!

```
1  if (x == 0)
2    if (y == 0) { error(); }
3  else {
4    f(x+y);
5  }
```

```
1  /* compiler's interpretation: */
2  if (x == 0) {
3    if (y == 0) {
4      error();
5    } else {
6      f(x+y);
7    }
8  }
```

Operator Precedence (1/2)

Precedence/associativity rules determine how the parser treats complex expressions

```
1 if (flag & FLAG != 0) { ... }   /* what you write */
2 if ((flag & FLAG) != 0) { ... } /* what you expect */
3 if (flag & (FLAG != 0)) { ... } /* what actually happens */
```

```
1 r = h<<4 + 1;       /* what you write */
2 r = (h<<4) + 1;     /* what you expect */
3 r = h << (4 + 1);   /* what actually happens */
```

Tip: make your code more explicit

- Better to have too many parentheses than too few.
- Often better to split into multiple statements
- Compile with `-Wall`

Operator Precedence Table[1]

Operator	Explanation
`()`	Function call
`[]`	Array indexing
`. ->`	Structure element selection
`++ --`	Postfix increment/decrement (e.g., x++)
`++ --`	Prefix increment/decrement (e.g., ++x)
`+ - ! ~`	Unary operators: plus/minus/logical Negation/bitwise negation
`(type)`	Type casts
`* &`	Dereferencing/Address operator
`sizeof`	Determining the memory consumption in byte
`* / %`	Multiplication/division/modulo
`+ -`	Addition/Subtraction
`<< >>`	Bitwise shift left/right

[1] https://en.cppreference.com/w/c/language/operator_precedence

Operator Precedence Table

Operator	Explanation
`< <= > >=`	Relational operators: smaller than/smaller or equal/greater than/greater or equal
`== !=`	Relational operators: equal to/not equal to
`&`	bitwise AND
`^`	bitwise exclusive OR
`|`	bitwise inclusive OR
`&&`	logical AND
`||`	logical OR
`?:`	Ternary operator
`=`	Assignment
`+= -= *=`	Ass. with addition/subtraction/multiplication
`/= %= &=`	Ass. with division/modulo/bitwise AND
`^= |=`	Ass. with exclusive OR/inclusive OR
`<<= >>=`	Ass. with bitweise shift left/right
`,`	Separation operator

Comma operator

Known as *separation* operator: `x, y`

1. Evaluates as y
2. But only after evaluating x and discarding the result

E.g., `int x = 0; int y = (x++, ++x);` $\rightarrow y = 2$

Part II

Semantic Pitfalls

Implicit Initialization (or lack thereof)

```c
int i;
int main(void) {
  int k;
  printf("%d\n", i); /* Ok */
  printf("%d\n", k); /* Undefined behaviour */
}
```

- Global and static variables: initialized with `0`
- No implicit initialization for other variables
- Most compilers just use whatever "value" is there in memory

Moral of the story

Initialize explicitly! Even global/static variables

Pointers and Arrays

- C treats an array as a contiguous memory block
- Array size:
  - C89: Fixed at compile time (constant expression)
  - C99: May be the result of an expression (e.g., `int x=10; int ar[x];`)
- Two "real" operations: i) Get size of array ii) Get address of `0`th element
- All other operations are realized with pointer operations
- Think of `x[y]` as an abbreviation for `*(x+y)`

```
1  int a[] = {1, 2, 3, 4, 5};
2
3  printf("%d\n", a[3]); /* OK */
4  printf("%d\n", 3[a]); /* OK! but strange... don't do this */
5
6  /* These statements are the same */
7  int main(int argc, char *argv[]) { ... }
8  int main(int argc, char **argv) { ... }
```

Arrays as Parameters


```
1  int foo(int x[]) { ... }
2
3  ar = {1, 2, 3, 4, 5};
4  foo(ar);
```

- Passing an array `ar` to a function
- Compiler automatically turns `ar` into a pointer to `ar[0]` (*decay*)
- The array is not copied!

Caution

In other contexts, `ar` is not the same as `&ar[0]`!

- `sizeof ar` = no. of bytes `ar` takes in memory
- `sizeof &ar[0]` = size of a memory address (e.g., 4 or 8 bytes)

Differences: Pointer and Arrays

```c
int a[] = {1,2,3};
int *p = a; /* Address of first element, same as &a[0] */
```

```c
printf("%d %d\n", a[2], p[2]); /* 3 3 */
```

```c
printf("%ld %ld %ld\n",
  sizeof(a), sizeof(p), sizeof(*p);
  /* on someone's pc: 12 8 4 */
```

```c
#define NRELEMENTS(a) (sizeof(a) / sizeof(a[0]))
/* Works for arrays, but not for pointers! */
```

Order of Evaluation[2]

Determines how the compiler generates code for a complex expression
Defined for four operators only

```
&&          ||          ?:          ,
```

Example: `a < b && c < d`

- `a < b` will definitely be evaluated before `c < d`
- Which is evaluated first, `a` or `b`? You cannot know in advance!

[2] https://en.cppreference.com/w/c/language/eval_order

Order of Evaluation: Pitfalls

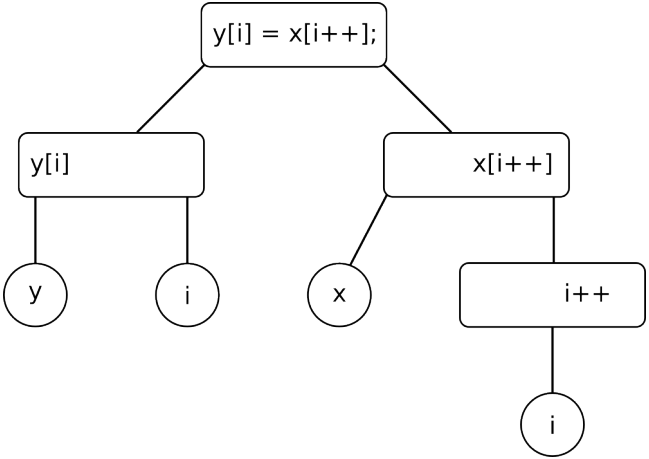

```
1  while (i < n)
2    y[i] = x[i++];
```

- Order of evaluation is not fixed, will be decided by the compiler
- Thought model: A tree is constructed with the help of precedence and associativity rules. But leaves may be evaluated in any order
- (Except leaves/subtrees of `&&`, `||`, `?:`, and `,` nodes)

```
1  /* What you expect: */
2  while (i < n) {
3    y[i] = x[i];
4    i++;
5  }
```

```
1  /* What might happen: */
2  while (i < n) {
3    tmp = x[i];
4    i++;
5    y[i] = tmp;
6  }
```

Thought Model




```
Load    tmp, x[i]
Store   tmp, y[i]
Add     i, i, 1
```

$a \prec b$ (a happens before b)

`i++` $\prec$ `x[i++]` ? ✓

`y[i]` $\prec$ `i++` ? ???

Dynamic memory allocation

- `malloc(size)` allocates memory
- `calloc(size)` allocates and initializes memory with zeroes
- `realloc(ptr, newSize)` changes the size of a memory block (may move it, so don't use the old `ptr` after!)

- All can return `NULL`, so always check before use!
- Pointer passed to `realloc`:
  - Must come from `malloc`/`calloc`/`realloc`
  - Must not have been `free`d before
  - `NULL` also allowed: behaves like `malloc`
  - Otherwise: undefined behavior (UB)

Bad operations usually lead to a segmentation fault (segfault)

- Attempt to access memory that was not given to the program
- OS kills the program in response

free

- `free` expects a pointer that was returned by `malloc`/`calloc`/`realloc`
- If you do pointer arithmetic, `free` might crash! (UB)
- Unless you bring the pointer back to the original address

```c
char *p = malloc(sizeof (*p) * 6); // 6 bytes
p += 3;
free(p); /* undefined behavior */
```

```c
char *p = malloc(sizeof (*p) * 6); // 6 bytes
p += 3;
p -= 2;
free(--p); /* works! */
```

Strings

- You can only assign `"literal"` to `char[]` at initialization
- Use string functions (`#include <string.h>`) for other operations

```
1  char buffer[256] = "init"; /* OK */
2
3  buffer = "hello";          /* Error */
4  strcpy(buffer, "hello");   /* OK */
```

- You can assign to `char*`

```
1  char *buf = "abc";
2  buf = "def"; /* OK */
```

- Compiler reserves memory for `"abc"` and `"def"`
- Initially `buf` points to `"abc"`
- After line 2, `buf` points to `"def"`

Important string functions

Function	Explanation
strlen(3)	Length of the string (excl. terminating '\0')
strstr(3)	Locates a substring
strcpy(3), strncpy(3)	Copies a string
strdup(3), strndup(3)	Duplicates a string
strcmp(3), strncmp(3)	Compares strings
strtol(3), strtoll(3)	String → integer conversion
strtof(3), strtod(3), strtold(3)	String → floating-point number conversion

- The number in brackets refers to the chapter in the man-page (e.g., `man 3 strlen`)

Fun with strings. . .


```
1 char str1[] = "text";
2 char *str2 = str1;
3 char *str3;
4 strcpy(str2, "more text");
5 strcpy(str3, "hello");
```

- Line 4: segfault because `str1` can only hold 5 characters (`text\0`)
- Line 5: segfault because `str3` is uninitialized (UB)

```
1 char *str = "constant";
2 strcpy(str, "hello"); /* segfault! But why? */
```

- We used `char *str` instead of `char str[]`
- String is stored in a read-only section of memory (pointed by `str`)
- You do not even get a warning!

Buffer overflows

- `strcpy` does not check the length of the destination buffer
- Malicious user may set `str_from_user` to a very long string
- Buffer overflow: write beyond the allocated memory (security risk)
- Solution: Use `strncpy` but add a terminating `\0`

```
1 char str[MAX_LENGTH];
2 strncpy(str, str_from_user, MAX_LENGTH - 1);
3 str[MAX_LENGTH - 1] = '\0';
```

From `man 3 strncpy`:

The `strncpy()` function is similar, except that at most n bytes of src are copied. Warning: If there is no null byte among the first n bytes of src, the string placed in dest will not be null terminated. (Emphasis mine)

Buffer overflows: Example


```
1  typedef struct player {
2    char name[8];
3    unsigned int gold;
4    unsigned char level;
5  } player_t;
6
7  player_t p = {.name = "", .level = 1, .gold = 0 };
8  strcpy(p.name, "alice"); // Benign user input
9  p.name[7] = '\0';
10 printf("Name: %s, level: %d, gold: %d\n", p.name, p.level, p.gold);
11 /* Output: Name: alice, level: 1, gold: 0 */
12
13 strcpy(p.name, "alicezzzzzzzzz"); // Malicious user input
14 p.name[7] = '\0';
15 printf("Name: %s, level: %d, gold: %d\n", p.name, p.level, p.gold);
16 /* Output: Name: alicezz, level: 122, gold: 2054847098 */
```

Part III

Lexical Pitfalls

Lexical Pitfalls

Lexical analysis: turn a sequence of characters into a sequence of tokens
In C, this happens twice

- Preprocessor (e.g., for macro replacements)
- Compiler itself (for actual parsing and compiling)

Multi Character Tokens

- Two or more characters may be separate tokens, or a single one
- Decided by the lexical analyzer, may have unexpected results

```
1 y = x/*p /* p points at divisor */;
2 /* Compiler interprets first "/*" as
3  * the beginning of a comment */
```

```
1 /* better */
2 y = x / *p  /* p points at divisor */;
3 y = x /(*p) /* p points at divisor */;
4 y = x / *p; /* comment after statement */
```

Macros


```
1  #define SQR(x) x*x
2
3  int a = 2;
4  int b = 2;
5
6  SQR(a);   /* 4 */
7  SQR(a+b); /* a + b * a + b == 8 */
8  SQR(a++); /* a++ * a++, UB!, a == 4 */
9
10 a = 2;
11 SQR(++a); /* ++a * ++a, UB!, a == 4 */
12 SQR((a+b)); /* (a+b) * (a+b) == 16 */
13
14 #define SafeSQR(x) (x)*(x)
15 SafeSQR(a+b); /* (a+b) * (a+b) == 16 */
16 SafeSQR(a++) /* (a++) * (a++), UB!, a == 4
17                 not so safe after all */
```

- Be careful with possibly undefined order of evaluations!
- Use brackets and try not to use ambiguous expressions

Macros: Multiple Statements


```
1  #define CMDS \
2    a = b; \
3    c = d;
4
5  if (var == 23)
6    CMDS
7  else
8    return;
```

Is converted to:

```
1  if (var == 23)
2    a = b;
3  c = d;
4  else /* syntax error */
5    return;
```

- Syntax error: `else` must immediately follow an `if`
- Without `else`: `c = d` is always executed!

Multi-statement macro

Please think twice

But if you are absolutely certain that you need a multi-statement macro:

```
1 #define CMDS \
2 do { \
3   a = b; \
4   c = d; \
5 } while(0)
```

- `do { ... } while(0)` is a single statement
- The body is executed exactly once (0 is guaranteed to be false)
- It works everywhere C expects a statement
- It avoids problems with `if/else` and other constructs
  (which you should always embrace anyway)